1 · Architecture and trust boundaries



Raw note → local redaction → cleaned-text API boundary.

One note, one evidence record — The browser runs int8 ONNX OpenMed models via transformers.js. **useSynthesis.start** loads models → deidentify(raw) → extractEntities(clean) → POST clean text, spans and redaction metadata. Model-load or inference failure stops before POST. The server constructs an ExtractionResult, then writers consume it; four roles are prompts over shared facts, not four separately trained models.

Data contracts — Entity = text, type, start/end offsets, confidence, source. Offsets refer to the cleaned note. Server checks note.slice(start,end) equals entity.text; that proves occurrence, not truth, negation or timing. "Surgery considered" must remain proposed. The API accepts client text, so the local workflow alone does not enforce de-identification for every caller.

Why local extraction — Redaction before transmission limits raw-note exposure and moves inference off the server. Costs: initial download, browser RAM/CPU, device variability. A missed identifier can remain in the "clean" string. Seeded recall is not proof that every real identifier is removed.

2 · Extraction: tokens → spans → assertions

BIO aggregation — openmed.ts joins B-/I- word pieces, strips ##, finds text with a forward cursor and assigns the **minimum token score** to the span. De-ID threshold defaults to 0.5; disease/pharma to 0.6. Replace spans right-to-left to preserve earlier offsets. Disease and pharma passes run in parallel; duplicate (start,end,type) spans retain the higher confidence.

```
for span in sorted(spans, start descending):
clean = clean[:start] + tag + clean[end:]
```

Where it breaks — Reconstructed offsets can fail on punctuation, repeated mentions or tokenizer normalization. The route drops nearby negated diagnosis/symptom spans, expands abbreviations and maps chemicals through RxNorm. Nearby regex negation is not full assertion analysis: uncertainty, historical findings and family history need separate status fields.

Better test — Use tokenizer offsets where possible; fixtures for repeated terms, "no X but Y," historical vs current conditions, and proposed vs performed services. Score exact/overlap spans separately. Measure extraction and assertion errors before blaming the writer.

3 · Small-model inference and feature parity

$$\text{TF-IDF: } x_{(j)} = (1 + \ln \text{count}_{(j)}) \cdot \text{IDF}_{(j)}; \text{ normalize } x/\text{L2norm}(x)$$
$$\text{logit}_{(c)} = b_{(c)} + w_{(c)}^{T}x; \; P(c|x) = \text{softmax}(\text{logits})_{(c)}$$

classifyNoteType — Unigrams+bigrams → exported vocabulary/IDF → sparse normalized vector → class logits → stable softmax (subtract max logit). Seven note types. JSON inference avoids Python/model-server overhead, but word/feature/class order must match training exactly.

detectMissing — One logistic classifier per field uses counts, lexical flags, normalized length and one-hot note type. **present=true means the missing-field flag is active** at p≥0.5, not that documentation is present. Lexical "left" or "mg" anywhere in a note is weak contextual evidence. Compare Python/TS logits on fixed fixtures; use per-field recall and thresholds.

4 · Retrieval, reranking and the neural option



Optional neural retrieval adds validated candidates; evidence remains necessary.

Lexical path — knowledge.server.ts lazily loads compressed official artifacts. icdCandidates expands abbreviations; inverted token postings require all query tokens, then back off to fewer/strong tokens. Rank by overlap, head match and fewer extras; validate and deduplicate codes. linkDiagnoses reranks **8 candidates** using [overlap, termLen, billable, rank], keeps one, abstains below **0.15**. Literal codes overwrite lexical links.

$$\text{Rerank score} = \sigma(b + w^{T}x), \; \sigma(z)=1/(1+e\text{^}(-z))$$
Candidate recall@k bounds what any reranker can recover.

Design tradeoff — A constrained catalog makes out-of-index output impossible; a valid code can still be the wrong code. Backoff improves recall but raises ambiguity. The lexical scorer does not read the full note. Measure retrieval recall@8, reranking accuracy and coverage after abstention separately.

Neural path — BiEncoder mean-pools **masked** token states and L2-normalizes B×D vectors. Code descriptions are precomputable; query→catalog dot products retrieve candidates. CrossEncoder jointly attends to query+description for a richer score, but costs one pass per pair. Runtime /code is conditional on SYNTHURE_MODEL_API, requests top 5, validates returned codes and falls back to no neural additions on failure.

$$\text{InfoNCE logits } S = QD^{T} / \tau; \; \text{loss} = \tfrac{1}{2}[\text{CE}(S,I)+\text{CE}(S^{T},I)]$$
Mask off-diagonal same-code pairs: synonyms are not negatives.

RAG distinction — Retrieval supplies external evidence to a writer; it does not train the writer or prove entailment. Here the lexical catalog route and optional neural coder are different retrieval mechanisms. Provenance must identify actual candidates/model/version, not just say "RAG."

5 · Readiness, calibration and review

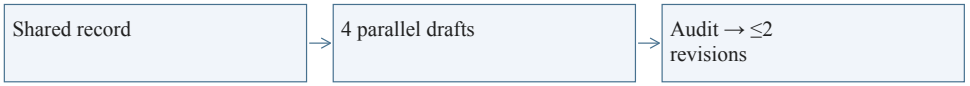
predictReadiness — Exported gradient-boosted trees traverse feature thresholds; leaf values add to initial log-odds, scaled by learning rate, then sigmoid. Isotonic knots map raw p to calibrated p. Bands: **<0.40 not ready; 0.40–<0.66 needs work; ≥0.66 ready**. These concern readiness labels, not a denial or payment probability.

$$s = \text{init} + \eta \, \Sigma_{(\text{trees})} \, \text{leaf}(x); \; p = \sigma(s)$$
Calibrated p = interpolation between stored isotonic knots.

Calibration is conditional — Fit mapping on separate data; evaluate reliability/ECE on held-out distribution. A good AUROC measures ordering, not probability accuracy. Missing evidence can block acceptance at any score. Below the first knot, this implementation can extrapolate; test boundaries and consider explicit clamping.

Knowledge and cost — ICD tabular validates codes; CCSR groups conditions; RxNorm checks drug vocabulary; CMS schedules support price illustrations; MedlinePlus adds consumer text with a 4-second timeout and per-instance cache. A fee schedule is not the patient's negotiated bill. Keep assumed deductible/coinsurance inputs visible; an inferred visit is a suggestion, not an observed procedure.

6 · Writers, audit and bounded revision



Provisional drafts stream first; final deterministic gate follows revisions.

route.ts control flow — After extraction, Promise.all launches patient, physician, hospital and employer drafts; each streams as it completes. Sonnet is selected only for hospital/physician in the frontier lane, otherwise Haiku. A forced report tool constrains structure. Combined audit waits for all drafts; flagged issues feed at most **two report revisions total**, then deterministic checks rerun.

Latency vs cost — Writer-phase time ≈ max(T1…T4), not ΣT_i; extraction, audit and revisions are serial additions. Parallel calls still incur token cost. One rejection fails Promise.all; independent failure isolation requires an explicit partial-output policy. Streamed drafts must remain provisional until final checks.

7 · Guardrails: evidence, policy, consistency

Display score = 1 − Σ(flagged severity weights)/Σ(all weights)
Decision precedence: block → escalate → revise → ship

runGuardrails — Any blocking finding wins. Otherwise extraction confidence <0.6 or no ICD/procedure codes escalates; medium/high findings request revision; otherwise ship. Layers cover input, grounding, policy, consistency, style and quality. A high average score cannot cancel a blocking finding.

What is not proven — Schema-valid JSON can be false. A number appearing in the source can be assigned to the wrong person or condition. Two agreeing model calls can repeat the same error. "Ship" is a software verdict, not downstream authorization or clinical validation. Separate supported facts, audience permission and action approval.

Stronger design — Use source-linked subject–predicate–value–status claims; test proposed→completed flips, wrong-person statistics and aggregate-view leaks. Keep human review for unsupported or high-risk outputs. Enforce gates at export/action boundaries, not just a badge in the UI.

8 · Streaming, cancellation and operations

```
buf += decoder.decode(chunk, stream=True)
while complete double-newline event exists:
parse data JSON; dispatch by event.type
```

Framing — A transport chunk may contain half an event or several events. useSynthesis buffers complete data frames and routes extracted/report/verification/safety/guardrail events. runId/alive() rejects late UI updates from superseded runs; it does **not** cancel the old HTTP request or model bill. The inspected fetch has no AbortSignal.

Production alternatives — Thread cancellation through client/server; bounded timeouts and retries; request-local errors; durable run/audit IDs. Rate-limit maps and caches are process-local, not shared across instances. Audit append is best-effort in the demo. Shared atomic quotas and durable audit storage are separate upgrades.

Measure the bottleneck — Split cold model download from warm NER, retrieval, writer fan-out, audit and revision. Record first-report latency, final-gate p95, provider errors and cost per accepted run. More agents or bigger models are not inherently better; ablate one writer vs four roles on the same inputs.

9 · Results: denominator and provenance first

Measure	Reported value / dataset
Lexical coding top-1 / top-3	74.1% / 81.6%; 727 synthetic cases
Neural acc@1 / acc@5 / MRR	40.66% / 49.18% / .4443; 3,615 CodiEsp mentions
Note type	100%; 360 synthetic; leakage caveat
Sections P / R / F1	1.000 / .696 / .821; synthetic
Missing-field micro-F1	.749; synthetic
Readiness AUROC / ECE	.839 / .054; 360 synthetic
Readiness accuracy	88.3% all → 91.7% at ~80% coverage
Seeded de-ID / disease / pharma recall	1.00 (90) / .83 (60) / .944 (60)
Adversarial cases caught	15/15 handcrafted deterministic cases

Comparability — These are ledger-reported results, not reruns. Synthetic and translated CodiEsp results are different tasks/distributions: 74.1 vs 40.66 is not a direct lexical-vs-neural comparison. Synthetic labels generated with the note can reward template recognition rather than real generalization.

Precision = TP/(TP+FP); Recall = TP/(TP+FN)
F1 = 2PR/(P+R); MRR = mean(1/rank of correct answer)

Concrete denominator — If 12 facts are present, 8 are found and 2 false ones added: precision=8/10=.80; recall=8/12=.667; F1≈.727. Perfect precision can coexist with missed facts. Report entity type, span matching policy and sample counts.

Typed evidence — Out-of-index rate 0 is **by construction**, not zero coding error. Real-note type accuracy, learned faithfulness AUROC, human fabrication rate, p95 and cost remain deferred/null in the ledger. Null means unmeasured. Top-level commit is null, so preserve the inspected repository revision with any exported results.

10 · What to defend and what to change

Architecture decision — Cheap local extraction and small exported classifiers produce an inspectable record; catalog retrieval constrains codes; hosted models narrate; deterministic and model checks govern review. Explain what each stage consumes/returns and why its failure cannot be repaired merely by better prose.

Most useful changes — Offset-aware extraction and assertion status; contextual candidate validation; grouped real-note holdouts; calibration boundary tests; per-stage failure states/cancellation; durable gate-enforced exports. Compare each against the current baseline rather than claiming an unmeasured improvement.

Reproduction checklist — Pin code/data/model artifacts and feature order. Verify TS/Python parity, reload artifacts, test absent fields/negation and provider failure, freeze train/calibration/test roles, then compare accepted quality *and* coverage. Measure resource/cost effects before scaling.

Source map — OpenMed: lib/openmed.ts; orchestration: lib/useSynthesis.ts; API: app/api/synthesize/route.ts; lexical knowledge: lib/knowledge.server.ts; predictors: lib/models/synthure.ts; neural coder: ml/icd_coder/model.py; gates: lib/guardrails.ts; metrics: data/evals.json. Blog sections 1–13 cover the full derivations and pinned code excerpts.